

NBA 5th Player Prediction

SOFE 4620U - Machine Learning and Data Mining

Emily Lai - 100825007

Jeremy Mark Tubongbanua - 100849092

Natasha Naorem - 100845321

GitHub Link

<https://github.com/JeremyTubongbanua/sofe4620u-nba>

Problem

Our goal is to develop a model that takes these input features, and makes a prediction on who is the 5th player that will optimize the outcome.

We decided to build an MLP (Multi-Layered Perceptron) to predict the 5th player.

Initial tensor design (rough work):

Input Tensors:

- season (e.g. 2007)
- home_team (e.g. "IND")
- away_team (e.g. "BOS")
- List of 4 home players
- List of 5 away players

Output Tensors:

- 5th player (tensor size will be # of unique names)

Pre-Processing Data

create_data.py

We developed a Python script that pre-processes all of the training data to our liking. The following sections outline what this script accomplishes.

Goal of the script: Encode the Strings into Dictionaries.. "Dictionary Encoding"

Input: aggregated matchups-20*.csv

Output: games.txt, names.txt, seasons.txt, teams.txt

Additional Outputs: mapped_data.csv and mapped_expanded.csv

Indexing

In order to pass the data from the CSVs into the model, we had to "index" the data (which is the process of converting class names to indices) in order to pass them through tensors.

The following text files were generated using a python script:

- games.txt - list of all unique games
- names.txt - list of all unique names
- seasons.txt - list of all unique seasons
- teams.txt - list of all unique teams

mapped_data.csv

We assumed that the position of players did not matter. For example, tuple (lebron james, jeremy lin, kobe bryant) is the same team as (jeremy lin, kobe bryant, lebron james). So we decided to aggregate the home_* and away_* columns and create sorted lists instead. A sorted list trivially ensures that the order doesn't matter for the same combination of players.

After indexing and aggregating players, the columns look like this:

game	season	home_team	away_team	starting_min	outcome	home_players	away_players
9610	8	29	10	0	-1	[88, 241, 256, 476, 897]	[51, 307, 396, 617, 958]
9610	8	29	10	7	-1	[88, 241, 256, 476, 897]	[307, 355, 396, 617, 958]
9610	8	29	10	8	-1	[138, 241, 256, 785, 897]	[43, 355, 617, 713, 958]
9610	8	29	10	9	-1	[138, 241, 256, 277, 785]	[43, 355, 617, 713, 958]
9610	8	29	10	10	-1	[138, 277, 785, 843, 859]	[43, 355, 617, 713, 958]

mapped_data.csv

mapped_expanded.csv

Once mapped_data.csv was generated, we generated mapped_expanded.csv, which is the core CSV file that is used in our model training.

Each row effectively became 5 more rows. One player is removed from the home players list and added to the player_removed column. This is done for all 5 players, one at a time.

By the end, this is what it looked like (1,184,560 rows):

game	season	home_team	away_team	starting_min	outcome	home_players	away_players	player_removed
9610	8	29	10	11	1	[277, 785, 843, 859]	[43, 355, 617, 652, 713]	138
9610	8	29	10	13	1	[277, 785, 843, 859]	[43, 51, 573, 652, 713]	138
9610	8	29	10	20	1	[88, 138, 256, 897]	[307, 396, 617, 795, 958]	241
9610	8	29	10	26	1	[88, 241, 256, 476]	[307, 355, 396, 617, 958]	897
9610	8	29	10	29	1	[138, 241, 785, 897]	[43, 307, 617, 795, 958]	476

mapped_expanded.csv

Model Design

Taking mapped_expanded.csv as our training data, we filter out only for the rows with outcome == 1. We only want to train the model on the rows and scenarios where the outcome was 1, which optimizes the home team for winning. Once filtered, we were cut down to 440,875 rows of

training data. After filtering the data, it just becomes a classification problem where the model has to predict which player was removed from the team.

We established the tensor shapes:

Input

- Season (8 → 2013)
- Home Team (29 → "PHO")
- Away Team (10 → "DEN")
- Home Players (length 4)
- Away Players (length 5)

6 tensors with shapes (1, 1, 1, 4, 5)

Target Output (what the model is predicting)

- Player_Removed

1 tensor with shape (1044)

Forward Pass Definition:

1. Embed the Season data, home_team and away_team data, and Player (home & away) data to size 64 vector.
2. Set up the (season, startming_min) as the context feature
3. Set up the (home_team and away_team) as a teams feature
4. Set up the home_players and away_players as their own features
5. Combine all features, apply ReLU and Dropout
6. Return Logits

Embedding Layers

Using Embeddings, we can represent complex characteristics in categorical data.

3 Embedding Layers:

- Season Embedding
- Player Embedding
- Team Embedding

Input

- Season (Pass into Season Embedding)
- Home Team (Pass into Team Embedding)
- Away Team (Pass into Team Embedding)
- Home Players (Pass into Player Embedding)
- Away Players (Pass into Player Embedding)

Output

- Season (vector size 64)
- Home Team (vector size 64)

- Away Team (vector size 64)
- Home Players
- Away Players

Embedded Layer Definitions

```
self.player_embedding = nn.Embedding(len(names), embedding_dim)
self.season_embedding = nn.Embedding(len(seasons), embedding_dim)
self.team_embedding = nn.Embedding(len(teams), embedding_dim)
```

Forward Pass

```
season_emb = self.season_embedding(season_idx)
home_team_emb = self.team_embedding(home_team_idx)
away_team_emb = self.team_embedding(away_team_idx)

home_player_embeddings = self.player_embedding(home_players)
home_player_embeddings = home_player_embeddings.view(batch_size, -1)

away_player_embeddings = self.player_embedding(away_players)
away_player_embeddings = away_player_embeddings.view(batch_size, -1)
```

Feature Processing Layers

Once our data is embedded, we use Feature Layers to transform them into higher-level features useful for predicting. We broke features up in order to achieve domain-specific patterns.

4 Feature Processing Linear Layers

- Home Players FC ($4 \times 64 \rightarrow 128$)
- Away Players FC ($5 \times 64 \rightarrow 128$)
- Teams FC ($2 \times 64 \rightarrow 128$)
- Context (season, starting_min) FC ($64+1 \rightarrow 128$)

Input

- 4×64 Home Players neurons (from the Embedding Layers)
- 5×64 Away Players neurons (from the Embedding Layers)
- 2×64 Team neurons (from the Embedding Layers)
- $64+1$ Context neurons (from the Embedding Layers)

Output

- 128 neurons representing the 4 home players
- 128 neurons representing the 5 away players
- 128 neurons representing home team and away team
- 128 neurons representing the (season, starting_min)

Feature Layer Definitions

```
self.home_players_fc = nn.Linear(embedding_dim * 4, hidden_dim)
self.away_players_fc = nn.Linear(embedding_dim * 5, hidden_dim)
self.teams_fc = nn.Linear(embedding_dim * 2, hidden_dim)
self.context_fc = nn.Linear(embedding_dim + 1, hidden_dim)
```

Forward Pass

```
# 2. Features
home_player_features = F.relu(self.home_players_fc(home_player_embeddings))
away_player_features = F.relu(self.away_players_fc(away_player_embeddings))

context_features = torch.cat([
    season_emb,
    starting_min.unsqueeze(1)
], dim=1)
context_features = F.relu(self.context_fc(context_features))

teams_features = torch.cat([
    home_team_emb,
    away_team_emb
], dim=1)
teams_features = F.relu(self.teams_fc(teams_features))
```

Combining Features Layer

This is the Combining Features Layer, where we combine all features to create an output.

Now that we have the following features from Step 2:

- Home Player Features (captures how the home players interact with each other) = 128 neurons
- Away Player Features (captures how the away players interact with each other) = 128 neurons
- Team Features (captures how the home_team interacts with the away_team) = 128 neurons

- Context Features (season and starting_min captures how the current time affects the model) = 128 neurons
- Total neurons = $128 * 4 = 512$

Combination Layer 1:

- We can capture the 512 neurons and break it down to 256 neurons.
- Apply ReLU (to allow the model to learn complex relationships)
- Apply Dropout (turns off 30% random neurons during training to prevent the model from overfitting)

Combination Layer 2:

- Capture 256 neurons and break it down to 128 neurons.
- Apply ReLU
- Apply Dropout

Output Layer:

- Capture 128 neurons and output to 1044 neurons

1044 neurons = 1044 unique names to which the model could turn on as the “predicted” player that was removed.

Combined Layer Definitions

```
self.combined_fc1 = nn.Linear(hidden_dim * 4, hidden_dim * 2)
self.combined_fc2 = nn.Linear(hidden_dim * 2, hidden_dim)
self.output_fc = nn.Linear(hidden_dim, len(names))
```

Forward Pass

```
# 3. Combined Features
combined_features = torch.cat([
    context_features,
    teams_features,
    home_player_features,
    away_player_features
], dim=1)

combined_features = F.relu(self.combined_fc1(combined_features))
combined_features = self.dropout(combined_features)
combined_features = F.relu(self.combined_fc2(combined_features))
combined_features = self.dropout(combined_features)

logits = self.output_fc(combined_features)
```

Decisions

Why 64 Embedded Dimensions and 128 Hidden Dimensions? Because: Too large = overfitting.
Too small = underfitting. Power of 2 can be efficient sometimes

Model Training

Multi Layered Perceptrons for classifications mostly use Cross Entropy Loss, Adam, and Batch Training.

Training Approach:

1. Forward Pass
2. Calculate the loss
3. Clear Gradients (optimizer.zero_grad())
4. Backward Pass
5. Optimizer Step (Apply gradients)

- **Optimizer:** Adam (lr=0.001) → Adaptive Moment Estimator is a popular optimizer, great at working with embeddings because it adapts its learning rate for each parameter
- **Criterion:** Cross Entropy Loss → good for classification problems
- **Num of Epochs:** 5
- **Batch Size:** 32 (we used batch training)
- **Train/Test Split Ratio:** 80%

With each epoch, we evaluated using an accuracy metric.

After 5 epochs of training, we achieved 37.18% testing accuracy. Not to be confused with NBA_test.csv.

Training Loop Code

```

1 def train_model(model, train_dataloader, test_dataloader, epochs, learning_rate, device):
2     loss_fn = nn.CrossEntropyLoss()
3     optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
4
5     train_losses = []
6     test_losses = []
7
8     for epoch in range(epochs):
9         model.train()
10        epoch_loss = 0
11
12        for batch_idx, (context, target) in enumerate(tqdm(train_dataloader, desc=f"Epoch {epoch+1}/{epochs}")):
13            context = context.to(device)
14            target = target.to(device).long().squeeze()
15
16            season_idx = context[:, 1]
17            home_team_idx = context[:, 2]
18            away_team_idx = context[:, 3]
19            home_players = context[:, 4:8]
20            away_players = context[:, 9:14]
21            starting_min = context[:, -2].float()
22
23            logits = model(season_idx, home_team_idx, away_team_idx, starting_min, home_players, away_players)
24            loss = loss_fn(logits, target)
25
26            optimizer.zero_grad()
27            loss.backward()
28            optimizer.step()
29
30            epoch_loss += loss.item()
31
32        avg_train_loss = epoch_loss / len(train_dataloader)
33        train_losses.append(avg_train_loss)
34
35        model.eval()
36        epoch_test_loss = 0
37        correct = 0
38        total = 0
39
40        with torch.no_grad():
41            for context, target in test_dataloader:
42                context = context.to(device)
43                target = target.to(device).long().squeeze()
44
45                season_idx = context[:, 1]
46                home_team_idx = context[:, 2]
47                away_team_idx = context[:, 3]
48                home_players = context[:, 4:8]
49                away_players = context[:, 9:14]
50                starting_min = context[:, -2].float()
51
52                logits = model(season_idx, home_team_idx, away_team_idx, starting_min, home_players, away_players)
53                loss = loss_fn(logits, target)
54
55                epoch_test_loss += loss.item()
56
57                _, predicted = torch.max(logits, 1)
58                total += target.size(0)
59                correct += (predicted == target).sum().item()
60
61        avg_test_loss = epoch_test_loss / len(test_dataloader)
62        test_losses.append(avg_test_loss)
63
64        accuracy = 100 * correct / total
65
66        print(f"Epoch {epoch+1}/{epochs}, Train Loss: {avg_train_loss:.4f}, Test Loss: {avg_test_loss:.4f}, Accuracy: {accuracy:.2f}%")
67
68    return train_losses, test_losses

```

Iteration 1

In our first iteration, we managed to achieve 37.18% accuracy over 5 epochs. This is good, but we wanted to try other things to hopefully achieve higher.

Some analytics:

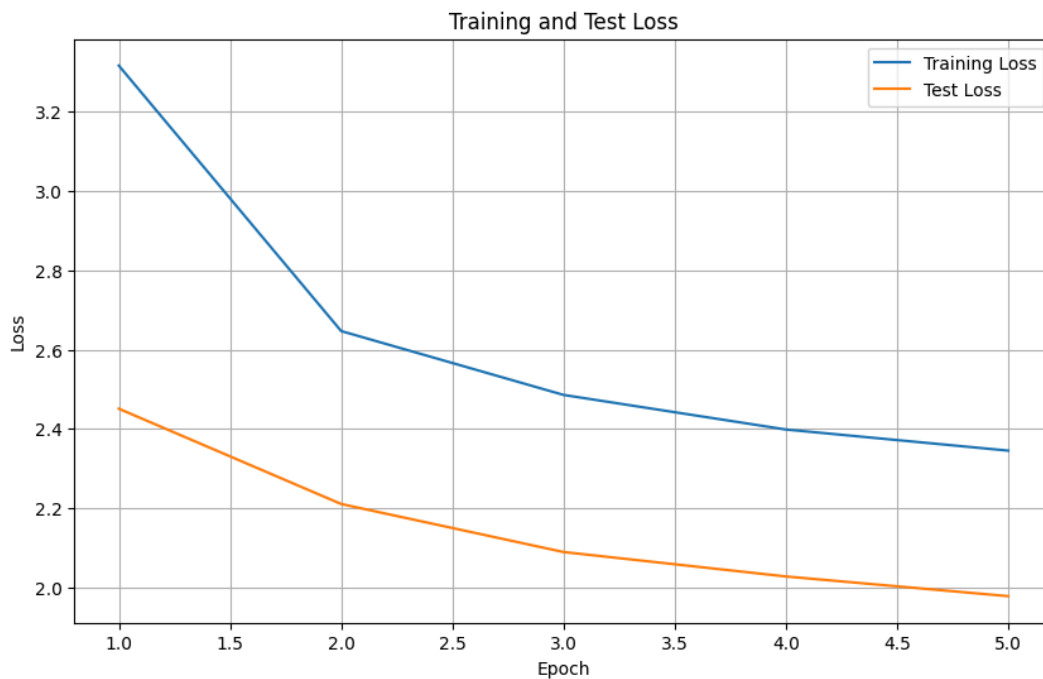
- At Epoch = 2, we see a substantial gap decrease
- Test loss and train loss continued to decrease over time

Results:

- Model training time: ~12min 45s
- Testing Accuracy: 37.18%
- Final Test Loss: 1.9789

```
No existing model found. Starting with a new model.  
Epoch 1/5: 100%|██████████| 11022/11022 [02:55<00:00, 62.84it/s]  
Epoch 1/5, Train Loss: 3.3154, Test Loss: 2.4511, Accuracy: 27.07%  
Epoch 2/5: 100%|██████████| 11022/11022 [02:48<00:00, 65.42it/s]  
Epoch 2/5, Train Loss: 2.6466, Test Loss: 2.2110, Accuracy: 32.83%  
Epoch 3/5: 100%|██████████| 11022/11022 [02:56<00:00, 62.50it/s]  
Epoch 3/5, Train Loss: 2.4858, Test Loss: 2.0900, Accuracy: 35.03%  
Epoch 4/5: 100%|██████████| 11022/11022 [02:57<00:00, 62.00it/s]  
Epoch 4/5, Train Loss: 2.3987, Test Loss: 2.0285, Accuracy: 35.86%  
Epoch 5/5: 100%|██████████| 11022/11022 [02:59<00:00, 61.23it/s]  
Epoch 5/5, Train Loss: 2.3455, Test Loss: 1.9789, Accuracy: 37.18%
```

Train Loop



Training and Testing Loss

Iteration 2

We tried doubling internal dimensions

- Embedding dimension 64 → 128
- Hidden dimension 128 → 256

Previous Results:

- Model training time: ~12min 45s
- Testing Accuracy: 37.18%
- Final Test Loss: 1.9789

Results:

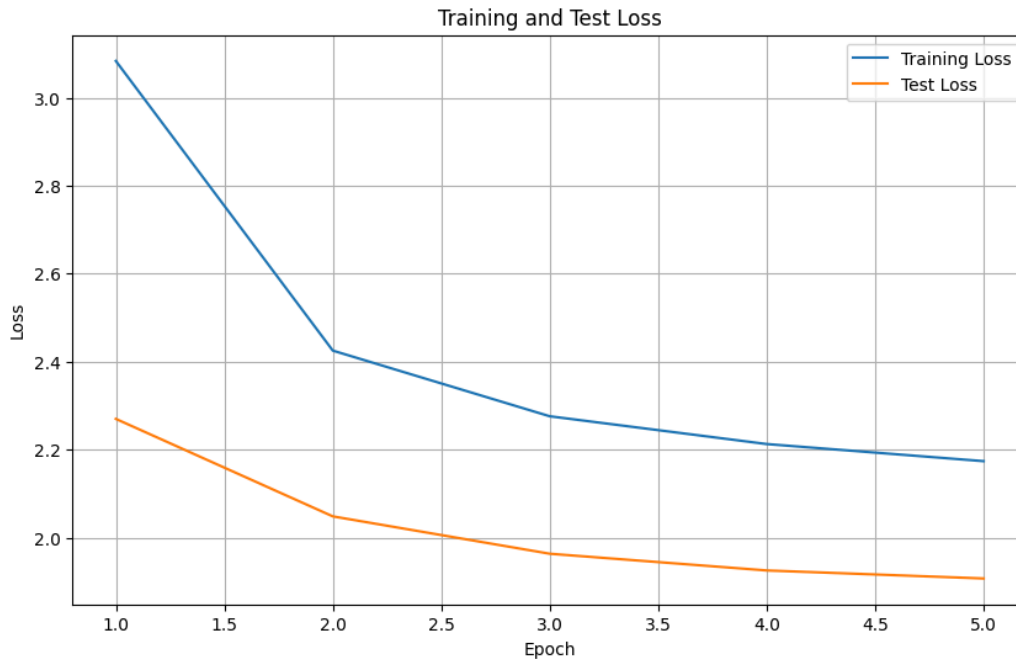
- Model training time: ~38 min
- Testing Accuracy: 37.83%
- Final testing loss: 1.9073

We increased the model training time from 12min45s to ~38 min, and only achieved a minor accuracy bump and a slightly lower test loss.

We concluded that this wasn't the solution.

```
Epoch 1/5: 100%|██████████| 11022/11022 [07:09<00:00, 25.65it/s]
Epoch 1/5, Train Loss: 3.0834, Test Loss: 2.2700, Accuracy: 31.05%
Epoch 2/5: 100%|██████████| 11022/11022 [07:29<00:00, 24.52it/s]
Epoch 2/5, Train Loss: 2.4252, Test Loss: 2.0485, Accuracy: 36.18%
Epoch 3/5: 100%|██████████| 11022/11022 [07:40<00:00, 23.91it/s]
Epoch 3/5, Train Loss: 2.2760, Test Loss: 1.9636, Accuracy: 37.18%
Epoch 4/5: 100%|██████████| 11022/11022 [07:57<00:00, 23.07it/s]
Epoch 4/5, Train Loss: 2.2128, Test Loss: 1.9255, Accuracy: 37.86%
Epoch 5/5: 100%|██████████| 11022/11022 [08:39<00:00, 21.21it/s]
Epoch 5/5, Train Loss: 2.1739, Test Loss: 1.9073, Accuracy: 37.83%
```

Training Loop Output



Training and Test Loss

Iteration 3

We adjusted the Embedding Dim and Hidden Dim back to 64 and 128. Then we decreased dropout from 0.3 to 0.25

Results:

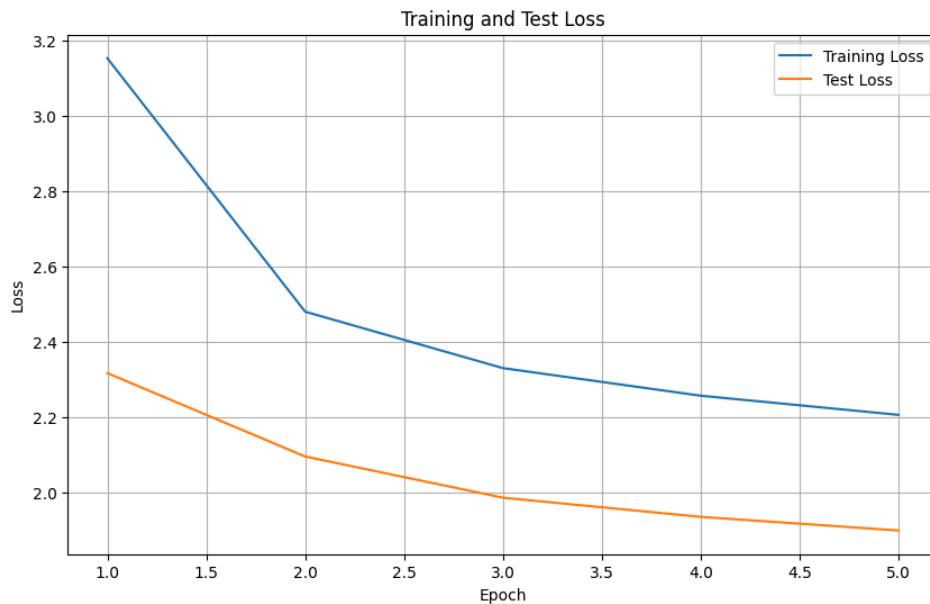
- Model training time: ~14min
- Testing Accuracy: 39.70%
- Final Test Loss: 1.8995

We were happy with the lower test loss.

We believe we could've stopped training at 3 epochs.

```
No existing model found. Starting with a new model.
Epoch 1/5: 100%|██████████| 11022/11022 [03:25<00:00, 53.69it/s]
Epoch 1/5, Train Loss: 3.1532, Test Loss: 2.3173, Accuracy: 30.55%
Epoch 2/5: 100%|██████████| 11022/11022 [03:24<00:00, 53.80it/s]
Epoch 2/5, Train Loss: 2.4804, Test Loss: 2.0959, Accuracy: 35.20%
Epoch 3/5: 100%|██████████| 11022/11022 [03:40<00:00, 50.03it/s]
Epoch 3/5, Train Loss: 2.3306, Test Loss: 1.9866, Accuracy: 37.88%
Epoch 4/5: 100%|██████████| 11022/11022 [03:39<00:00, 50.32it/s]
Epoch 4/5, Train Loss: 2.2573, Test Loss: 1.9356, Accuracy: 38.89%
Epoch 5/5: 100%|██████████| 11022/11022 [03:46<00:00, 48.57it/s]
Epoch 5/5, Train Loss: 2.2065, Test Loss: 1.8995, Accuracy: 39.70%
```

Training Loop



Training and Test Loss

Model Testing

Using our 3rd iteration of the model, we achieved the following results:

- 28.90% overall accuracy → The model successfully guessed the right 5th player 28.90% of the time.
- The 5th player was in the Top 10 candidates 73.00% of the time
- The 5th player was in the Top 5 candidates 65.30% of the time
- The 5th player was in the Top 3 candidates 53.90% of the time

See `NBA_Testing.ipynb`. This jupyter notebook created a final CSV of all predictions.

predicted_player	confidence	true_player	true_player_rank	true_player_probability	correct
Jermaine O'Neal	36.40018701553345	Troy Murphy	3.0	14.056769371032715	False
Chuck Hayes	25.059589743614197	Chuck Hayes	1.0	25.059589385986328	True
Manu Ginobili	22.772815823554993	Brent Barry	2.0	18.929443359375	False
Kevin Garnett	35.5131596326828	Craig Smith	3.0	22.162633895874023	False
Darko Milicic	96.0522711277008	Stromile Swift	14.0	0.002492165192961693	False
Charlie Bell	27.6822030544281	Charlie Bell	1.0	27.68220329284668	True
Dwyane Wade	26.970288157463074	Dwyane Wade	1.0	26.970287322998047	True
Anderson Varejao	72.23473787307739	Anderson Varejao	1.0	72.2347412109375	True
Stephen Jackson	31.178978085517883	Mickael Pietrus	3.0	12.622146606445312	False

Sample columns from generated CSV

Observation During Testing

- There are 40 new names in testing data in NBA_test.csv. The model has no prior data to these 40 new names, so it cannot make guesses on if it is good with other team members or if it should be a 5th player.
- There is a 10th season: NBA_test.csv has a 10th season: Season 2016. There is no past data on 2016, so predictions on 2016 games are unpredictable on who the model will recommend as the 5th player.
- 2 names that are not in the training data or the test data. NBA_test_labels.csv introduces two new names: “Anthony Brown” and “R.J. Hunter”, which are not found anywhere in `NBA\_test.csv` and `matchups-20\*.csv`. It is impossible for the model to ever get this prediction right, because it is a completely new name, as if they weren't even NBA players.

List of 40 names found in NBA_test.csv but did not exist in matchups-20*.csv

```
names: ['Alan Williams', 'Larry Nance', 'Bobby Portis', 'Nikola Jokic', 'Christian Wood', 'Norman Powell', 'Yus Jones', 'Frank Kaminsky', 'Devin Booker', 'Cameron Payne', 'Kristaps Porzingis', 'Josh Richardson', 'Emanuel Mudiay', 'Nemanja Bjelica', 'Stanley Johnson', 'Shabazz Muhammad', 'Angelo Russell', 'Willie Cauley-Stein', 'Earl Anthony Young', 'Justise Winslow', 'T.J. McConnell', 'Myles Turner', 'Jordan McRae', 'Terry Rozier', 'Marcelo Huertas', 'Jonathan Simmons', 'Hondae Hollis-Jefferson', 'Mario Hezronja', 'Montrezl Harrell', 'Willie Reed', 'Lamar Patterson', 'Xavier Munford', 'Kelly Oubre', 'Raul Neto', 'Richaun Holmes', 'Trey Lyles', 'Chris McCullough', 'Rashad Vaughn', 'DeLon Wright', 'Jerian Grant']
total unique names: 1884 | First 5 names: ['A.J. Price', 'Aaron Brooks', 'Aaron Gordon', 'Aaron Gray', 'Aaron McKie']
```

Why We Think our Prediction Accuracy is Low

1. On row 941: Our model was 44.44% confident that “Jordan Farmer” was the 5th optimal player, but the true answer was Anthony Brown. Since Anthony Brown's name is not an output neuron at all (since his name does not exist in any of the training data), this was a failed attempt from the start.

Jordan Farmer	44.448599219322205	Anthony Brown		
---------------	--------------------	---------------	--	--

2. On row 910: Our model was 52.69% confident that “Luke Ridnour” was the 5th optimal player, but the true answer was Shabazz Muhammad. It was completely wrong because the Away Team included Alan Williams which was on of the 40 names that was not included in the Training Data.

Luke Ridnour	52.69308090209961	Shabazz Muhammad	421.0	2.7463205320987782e-12	False
--------------	-------------------	------------------	-------	------------------------	-------

['Alan Williams', 'Chase Budinger', 'Devin Booker', 'Jon Leuer', 'Ronnie Price']

3. Training data only contained seasons 2007-2015. Testing data contained season 2016. The true player ranking standard deviation is substantially higher in 2016 than in the other seasons. This is expected, since the data our model was trained on is only for season 2007-2015

```
1 season_std_devs = results_df.groupby('season')['true_player_rank'].std()
2 print("Standard Deviation of True Removed Player Rank by Season:")
3 print(season_std_devs)
```

Standard Deviation of True Removed Player Rank by Season:

season	
2007	43.070724
2008	30.179262
2009	9.848222
2010	28.969570
2011	38.944367
2012	83.959015
2013	47.577317
2014	18.266874
2015	32.809304
2016	216.385154

Name: true_player_rank, dtype: float64

Future Improvements

Model doesn't take into account if the predicted home_player is part of the home_team!

- An improvement to this is to build a dictionary of valid answers, then have the model choose from the list of valid players.
- But we believe that since the model is trained on past data, that it predicts based on who is part of the valid team anyways, just by nature. To elaborate, the model is trained on data from 2013 games, for example, and therefore is trained to make predictions on the 5th player based on the 2013 games. So if you gave it a home team of 4 players from the 2013 season, it is trained to pick the 5th player based on who was on the 2013 season anyways.